

InsightFlow Agent 开源参考项目文档

1. 文档目标

本文档用于整理 InsightFlow Agent 开发过程中可以参考的开源项目，并明确每个项目应该参考什么、不应该参考什么。

InsightFlow Agent 的核心目标是突出：

- Multi-Agent
- Tool Calling
- SQL Execution
- SQL Execution Feedback Repair
- Trace
- Eval
- Report Generation
- 后续 Action Workflow

因此，参考项目不能只看“是否也是 Text2SQL”，而要看它是否能帮助我们实现以下能力：

- LangGraph workflow
- 多 Agent 节点拆分
- SQL 生成
- SQL 校验
- SQL 执行
- SQL 错误修复
- 图表生成
- 报告生成
- Eval
- 前端可视化展示
- 工程结构

最终建议：

- 主参考 1: [adamfaik/sql-agent](#)
- 主参考 2: [mallahyari/langgraph-sql-agent](#)
- 局部参考: [azain47/Multi-Agent-Text2SQL-System](#)

2. 最终推荐结论

2.1 最适合开发参考的两个项目

排名	项目	推荐定位
1	adamfaik/sql-agent	P0 主参考：SQL 执行、自修复、Streamlit、Eval
2	mallahyari/langgraph-sql-agent	P1 / 工程参考：多 Agent 拆分、图表、FastAPI、前端展示

2.2 辅助参考项目

项目	推荐定位
azain47/Multi-Agent-Text2SQL-System	局部参考：SQL validation、feedback loop、Feedback Formatter

2.3 不建议的做法

不建议同时深挖太多项目，也不建议把多个项目的结构混在一起。

推荐策略是：

P0：以 adamfaik/sql-agent 为主
P1：以 mallahyari/langgraph-sql-agent 为主
SQL Validator：局部参考 azain47
P2 / P3：在自己项目架构上扩展，不照搬任何一个项目

3. 主参考项目一：adamfaik/sql-agent

3.1 项目定位

adamfaik/sql-agent 是一个基于 LangGraph 的 self-correcting Text-to-SQL Agent，使用 SQLite 和 Streamlit，面向 Olist 电商数据集。

它非常适合作为 InsightFlow 的 P0 主参考。

P0 阶段我们要做的是：

Schema Tool
Metric Tool
SQL Generator Agent
SQL Reviewer Agent
SQL Executor Tool
Error Fix Agent
Insight Agent

Trace
Eval
Streamlit Demo

adamfaik/sql-agent 正好覆盖其中最核心的部分：

LangGraph 状态机
SQLite SQL 执行
SQL 执行错误反馈
失败后重新生成 SQL
Streamlit glass-box 展示
Eval benchmark
电商业务场景

3.2 为什么它适合做 P0 主参考

InsightFlow P0 的核心不是图表，也不是周报，而是：

Agent 生成 SQL
-> 工具校验 SQL
-> 工具真实执行 SQL
-> 数据库返回 rows 或 error
-> Agent 根据 error 修复 SQL
-> 再次执行

adamfaik/sql-agent 的项目目标和这个非常接近。

它的优势：

1. 有真实 SQLite 执行，不只是生成 SQL 文本。
2. 有 self-correction 设计，适合参考 SQL 错误修复。
3. 有 Streamlit 前端，适合参考 MVP Demo 展示。
4. 有 eval 目录和 benchmark，适合参考 AgentEval。
5. 使用真实电商数据集，和 InsightFlow 的电商场景接近。

3.3 具体参考点

3.3.1 LangGraph Workflow

可以参考它如何把 Text-to-SQL 拆成多个节点。

可以映射到 InsightFlow：

adamfaik/sql-agent	InsightFlow 对应模块
query reformulation	Supervisor Agent / Query Rewriter
query classification	Supervisor Agent
SQL generation	SQL Generator Agent
SQL execution	run_sql()
error handling / retry	Error Fix Agent
result summarization	Insight Agent

InsightFlow 不需要完全照搬它的节点名称，但可以参考它的流程思路：

```

用户问题
-> 查询改写 / 分类
-> SQL 生成
-> SQL 执行
-> 如果失败，把 error 写回 State
-> 再次生成 / 修复 SQL
-> 总结结果

```

3.3.2 SQL 执行工具

重点参考：

```

SQLite 连接
SQL 执行
timeout
row limit
错误捕获
结果结构化返回

```

InsightFlow 的 `run_sql()` 可以借鉴这种思路，但要更规范地拆成独立工具文件：

```
tools/sql_executor.py
```

推荐输出格式：

```

{
  "success": true,
  "columns": ["product_name", "sales"],
  "rows": [
    ["Camera A", 128000],
    ["Lens B", 113500]
  ]
}

```

```
],
"row_count": 2,
"execution_time_ms": 34
}
```

失败时：

```
{
  "success": false,
  "error": "no such column: oi.price",
  "execution_time_ms": 11
}
```

3.3.3 SQL 错误修复

这是 InsightFlow P0 最重要的技术卖点。

可以参考它的 self-correction 思路：

```
SQL 执行失败
-> 捕获数据库 error
-> error 写回 State
-> 下一轮 SQL 生成 / 修复时把 error 注入上下文
-> 重新执行
```

InsightFlow 要在这个基础上做得更清楚：

```
SQL Generator Agent
-> SQL Reviewer Agent
-> SQL Executor Tool
-> Error Fix Agent
-> SQL Reviewer Agent
-> SQL Executor Tool
```

也就是说，InsightFlow 不只是“重新生成 SQL”，而是明确有一个：

```
Error Fix Agent
```

它输入：

```
{
  "user_question": "...",
  "failed_sql": "...",
}
```

```
"error_message": "no such column: oi.price",
"database_schema": {...}
}
```

输出：

```
{
  "fixed_sql": "...",
  "fix_reason": "order_items 表中不存在 price 字段，应使用 unit_price。",
  "retry_count": 1
}
```

3.3.4 Streamlit Glass-box Demo

可以参考它的 Streamlit 展示方式。

InsightFlow 的 Streamlit 页面应该展示：

```
用户问题
当前 Agent 节点
生成 SQL
SQL Review 结果
SQL 执行结果
错误修复过程
最终回答
Trace
Eval 入口
```

不要只展示最终回答。

项目要让面试官看到：

```
Agent 到底调用了哪些工具
每个工具返回了什么
失败时系统怎么修复
最终答案是否基于 execution_result
```

3.3.5 Eval Benchmark

可以参考它把测试问题做成 benchmark 的方式。

InsightFlow P0 应该有：

```
eval/test_questions.json
eval/run_eval.py
eval/report.md
```

P0 测试集建议覆盖：

```
基础 SQL 查询
销售额排名
品类统计
时间趋势
SQL 错误修复
危险 SQL 拦截
指标口径校验
```

核心指标：

```
SQL execution success rate
SQL repair success rate
dangerous SQL block rate
metric definition accuracy
average tool calls
average latency
failure type distribution
```

3.4 不应该照搬的地方

`adamfaik/sql-agent` 更像一个集中的 self-correcting SQL Agent 项目，代码模块化程度不一定完全符合 InsightFlow 的目标。

InsightFlow 不应该照搬：

```
把所有逻辑集中放在 agent.py
只做 Text-to-SQL
只做一个 agent 的自修复
缺少明确 Agent / Tool 分层
缺少 Report / Evidence / Action 的后续扩展结构
```

InsightFlow 应该拆成：

```
agents/
tools/
graph/
eval/
```

```
reports/  
tests/
```

也就是说：

借鉴它的 P0 执行与修复闭环
但不要照搬它的项目结构

3.5 在 InsightFlow 中的使用方式

P0 重点参考

LangGraph 基础 workflow
SQL 执行
SQL 失败重试
Streamlit 展示
Eval benchmark

不参考

完整项目结构
P1 图表报告设计
P2 行动型 workflow
P3 工程化架构

4. 主参考项目二：mallahyari/langgraph-sql-agent

4.1 项目定位

`mallahyari/langgraph-sql-agent` 是一个基于 LangGraph 的 SQL 分析与可视化 Agent 项目。

它的特点是：

Router
Query Rewriter
Table Selector
SQL Generator
SQL Validator
SQL Executor
Response Synthesizer
Visualization Planner
Visualization Generator

FastAPI backend
React frontend
SSE streaming

它非常适合作为 InsightFlow 的 P1 / 工程架构参考。

4.2 为什么它适合做第二主参考

InsightFlow P1 要从 P0 的 Agentic SQL Core 升级到：

可靠业务分析
图表生成
Markdown 报告
Trace 展示
更清晰的多 Agent 模块化

`mallahyari/langgraph-sql-agent` 正好可以参考：

多 Agent 节点拆分
Table Selector
SQL Validator / Executor 条件边
Visualization Planner
Visualization Generator
FastAPI + React + SSE 展示
backend / frontend 分层
state / graph / agents / tools 分层

4.3 具体参考点

4.3.1 Multi-Agent 模块拆分

它的 specialized agents 很适合参考。

可以映射到 InsightFlow：

mallahyari/langgraph-sql-agent	InsightFlow 对应模块
Query Router	Supervisor Agent
Query Rewriter	Query Rewriter，可选
Table Selector	Schema Agent / Table Selector
SQL Generator	SQL Generator Agent
SQL Validator	SQL Reviewer Agent

mallahyari/langgraph-sql-agent	InsightFlow 对应模块
SQL Executor	run_sql()
Response Synthesizer	Insight Agent
Visualization Planner	Chart Agent
Visualization Generator	generate_chart()

InsightFlow P0 可以先不做 Table Selector，P1 再补。

4.3.2 LangGraph 条件边

可以参考它的 retry 逻辑：

```
SQL Validator invalid -> 回到 SQL Generator
SQL Executor error -> 回到 SQL Generator
SQL Executor success -> Response Synthesizer
Response Synthesizer -> Visualization Planner
Visualization Planner needs chart -> Visualization Generator
```

InsightFlow 应该改成：

```
validate_sql failed -> SQL Generator
run_sql failed and retry_count < 1 -> Error Fix Agent
run_sql success -> Insight Agent
Insight Agent -> Evidence Validator
Evidence Validator -> Chart Agent / Report Agent
```

区别在于：

```
mallahyari 更偏 “重新生成”
InsightFlow 应该显式做 “Error Fix Agent”
```

4.3.3 Visualization Planner

P1 图表生成可以参考它的可视化规划思路。

InsightFlow 的 Chart Agent 可以这样做：

```
排名问题 -> bar chart
趋势问题 -> line chart
```

占比问题 -> pie chart
下降分析 -> line chart + bar chart

Chart Agent 不直接画图，而是输出：

```
{  
  "need_chart": true,  
  "chart_type": "line",  
  "x": "month",  
  "y": "gmV",  
  "group_by": "category_name",  
  "reason": "用户询问最近 3 个月销售额下降趋势，需要折线图展示变化。"  
}
```

然后调用工具：

```
generate_chart()
```

4.3.4 前后端分层

如果 P0 用 Streamlit，P1/P3 可以参考它的工程结构：

```
backend/  
frontend/  
state/  
graph/  
agents/  
tools/
```

P3 如果要做 FastAPI + React + SSE，可以参考它的结构。

InsightFlow 的 P0 不建议一开始做 React，因为会拖慢进度。

推荐路径：

```
P0: Streamlit  
P1: Streamlit + 图表报告  
P3: FastAPI + React / SSE
```

4.3.5 Thinking Process / Streaming 展示

它的 SSE thinking process 对 InsightFlow 后续 Trace Dashboard 很有参考意义。

InsightFlow 可以先在 Streamlit 里展示：

Agent steps
Tool calls
SQL
Review result
Execution result
Retry path

P3 再做：

SSE streaming
Trace Dashboard
Run status API

4.4 不应该照搬的地方

`mallahyari/langgraph-sql-agent` 的 SQL Validator 相对偏简单，主要适合作为 workflow 参考。

InsightFlow 不应该只做：

regex forbidden keyword check

InsightFlow 的 SQL Reviewer 应该更强：

SELECT-only
禁止多语句
表名存在
字段存在
JOIN key 合理
时间范围符合问题
业务指标口径正确
敏感字段拦截
LIMIT 检查

另外，InsightFlow P0 不建议一开始照搬它的 React + FastAPI + SSE 架构，因为会让 MVP 变重。

4.5 在 InsightFlow 中的使用方式

P1 重点参考

多 Agent 模块拆分
Table Selector
Visualization Planner
Visualization Generator
Graph 条件边
前端展示思路

P3 可参考

FastAPI backend
React frontend
SSE streaming
工程结构

不参考

SQL Validator 实现细节
一开始就做前后端分离
完整 UI 复杂度

5. 局部参考项目：azain47/Multi-Agent-Text2SQL-System

5.1 项目定位

`azain47/Multi-Agent-Text2SQL-System` 是一个基于 LangGraph 的 Multi-Agent Text2SQL 系统。

它有比较清楚的 feedback loop:

Prompt Processor
-> Relevance Checker
-> Prompt Optimizer
-> SQL Generator
-> SQL Validator
-> Query Evaluator
-> Feedback Formatter
-> SQL Generator
-> Finalizer

它适合作为 InsightFlow 的局部参考，尤其是：

```
SQL validation
feedback loop
Feedback Formatter
状态循环
```

但不建议作为主开发模板。

5.2 为什么不作为主参考

它的问题是：

```
没有完整真实数据库执行器作为核心闭环
更偏 SQL 生成与验证
没有 Streamlit Demo
没有图表 / 报告生成
没有电商业务数据闭环
没有行动型 workflow
```

而 InsightFlow P0 最核心的是：

```
validate_sql()
-> run_sql()
-> 数据库返回真实 rows/error
-> Error Fix Agent 修复
```

所以它可以参考，但不能作为主模板。

5.3 具体参考点

5.3.1 SQL Validator

可以参考它使用 SQL parser / SQL AST 的思路。

InsightFlow 的 `validate_sql()` 建议结合：

```
sqlglot / sqlparse
schema dict
business metric definitions
sensitive field rules
```

检查：

是否 SELECT
是否多语句
表是否存在
字段是否存在
alias 是否正确
是否访问敏感字段
是否符合指标口径

5.3.2 Feedback Formatter

它的 Feedback Formatter 思路适合 InsightFlow 的 Error Fix Agent。

当 SQL 不通过或执行失败时，不要只把 error 原样塞给 LLM，而是格式化为明确修复提示：

```
{  
  "error_type": "COLUMN_NOT_FOUND",  
  "failed_column": "oi.price",  
  "table_schema": {  
    "order_items": ["id", "order_id", "product_id", "quantity", "unit_price"]  
  },  
  "repair_hint": "order_items 表中不存在 price 字段，可能应使用 unit_price。"  
}
```

这样 Error Fix Agent 会更稳定。

5.3.3 Iterative Feedback Loop

可以参考它的循环控制：

```
max_feedback_loops  
feedback_history  
final verdict  
failure message
```

InsightFlow P0 要更克制：

```
max_retry = 1
```

P1/P2 再考虑放宽到 2-3 次。

5.4 在 InsightFlow 中的使用方式

只参考三个点：

sqlglot-based SQL validation
Feedback Formatter
max retry / feedback history

不参考：

整体项目结构
没有 executor 的部分
最终 answer 逻辑
前端
报告
行动闭环

6. 三个项目对比

维度	adamfaik/sql-agent	mallahyari/langgraph-sql-agent	azain47/Multi-Agent-Text2SQL-System
是否适合做主参考	是，P0 主参考	是，P1 / 工程参考	否，局部参考
LangGraph	有	有	有
Multi-Agent 拆分	中等	强	强
真实 SQL 执行	有	有	不完整 / 待补
SQL 错误修复	强	有 retry，但不如 adamfaik 聚焦	有 feedback loop，但缺 executor
Streamlit	有	无，使用 React	无
图表生成	有一定参考	强	弱
报告生成	弱	弱	弱
Eval	强	一般	一般
工程结构	中等	强	中等
适合阶段	P0	P1 / P3	SQL Validator 局部
与 InsightFlow 贴合度	最高	高	中等

7. InsightFlow 应该如何吸收这些项目

7.1 P0：以 adamfaik/sql-agent 为主

P0 目标：

Multi-Agent + Tool Calling + SQL Execution Feedback

参考内容：

LangGraph 执行闭环
SQLite SQL 执行
错误反馈修复
Streamlit glass-box 展示
Eval benchmark

InsightFlow P0 实现：

Schema Agent
Metric Agent
SQL Generator Agent
SQL Reviewer Agent
SQL Executor Tool
Error Fix Agent
Insight Agent
Trace Logger
Eval

7.2 P0 SQL Validator：局部参考 azain47

P0 的 `validate_sql()` 可以参考 azain47 的 parser-based validation 思路。

实现时建议：

sqlparse 负责基础语句检查
sqlglot 负责 AST 解析和表字段提取
schema dict 负责表字段验证
metrics.yaml 负责业务口径验证
sensitive_fields.yaml 负责敏感字段拦截

7.3 P1：以 mallahyari/langgraph-sql-agent 为主

P1 目标：

Reliable Analysis + Report Generation

参考内容：

Table Selector
Visualization Planner
Visualization Generator
Graph 条件边
前端展示 agent steps

InsightFlow P1 实现：

Evidence Validator
Chart Agent
Report Agent
generate_chart()
save_report()

7.4 P2：自己实现，不照搬

P2 的经营周报和行动闭环，目前三个参考项目都没有完全覆盖。

InsightFlow 需要自己设计：

Report Supervisor Agent
Business Review Report
Action Planner Agent
Risk Assessor Agent
Approval Gate
create_task()
create_metric_alert()
Action Verifier
Audit Log

参考方式：

沿用 P0/P1 的 LangGraph workflow 和 tool calling 思路
不要照搬外部项目

7.5 P3: 部分参考 mallahyari

P3 做工程化时，可以参考 mallahyari 的：

```
FastAPI backend
React frontend
SSE streaming
frontend/backend 分离
```

但 P3 不是 MVP，不要提前做。

8. 推荐阅读顺序

第一步：读 adamfaik/sql-agent

优先读：

```
README.md
agent.py
app.py
eval/
scripts/
```

重点问题：

```
LangGraph State 怎么定义？
SQL 执行失败后怎么重试？
SQL error 怎么写回 prompt？
Streamlit 怎么展示 SQL 和错误修复？
eval benchmark 怎么组织？
```

第二步：读 mallahyari/langgraph-sql-agent

优先读：

```
README.md
SYSTEM_DESIGN.md
backend/app/graph/
backend/app/state/
backend/app/agents/
```

backend/app/tools/
frontend/

重点问题：

Router / Table Selector / SQL Generator / Validator 怎么拆？
Visualization Planner 怎么判断是否需要图表？
Graph 条件边怎么写？
SSE 怎么展示 agent steps？
前后端怎么分层？

第三步：只读 azain47 的关键部分

优先读：

README.md
agents/graph.py
utils/sqlvalidator.py
Feedback Formatter 相关代码
state / configuration

重点问题：

SQL validator 怎么做？
feedback loop 怎么组织？
max retry 怎么控制？
错误反馈怎么格式化给 SQL Generator？

9. 最终开发参考策略

9.1 不要照搬任何一个项目

InsightFlow 的目标比单纯 Text2SQL 更完整：

Agentic SQL Core
Reliable Analysis
Report Generation
Business Review Report
Action Workflow
MCP / Engineering

现有项目都只能覆盖一部分。

因此策略是：

参考架构，不复制架构
参考工具，不复制工具
参考流程，不复制项目定位

9.2 最终组合

P0 主体: adamfaik/sql-agent
P0 Validator: azain47/Multi-Agent-Text2SQL-System
P1 图表与工程结构: mallahyari/langgraph-sql-agent
P2 周报与 Action: InsightFlow 自己设计
P3 API / SSE: mallahyari/langgraph-sql-agent

9.3 最终参考边界

模块	参考项目	是否照搬
LangGraph 基础闭环	adamfaik/sql-agent	不照搬，参考流程
SQL Executor	adamfaik/sql-agent	可高度参考，但重写为工具
Error Fix Loop	adamfaik/sql-agent	参考思路，显式拆出 Error Fix Agent
Streamlit Demo	adamfaik/sql-agent	可参考展示方式
Eval Benchmark	adamfaik/sql-agent	参考结构，测试集自定义
SQL Validator	azain47	参考 parser-based 思路
Feedback Formatter	azain47	参考错误格式化思路
Multi-Agent 分层	mallahyari/langgraph-sql-agent	参考结构
Visualization Planner	mallahyari/langgraph-sql-agent	参考决策逻辑
FastAPI / SSE	mallahyari/langgraph-sql-agent	P3 再参考
Report Agent	自己设计	不照搬
Evidence Validator	自己设计	不照搬
Action Workflow	自己设计	不照搬

10. 最终结论

InsightFlow 最适合参考的两个主项目是：

1. adamfaik/sql-agent
2. mallahyari/langgraph-sql-agent

其中：

adamfaik/sql-agent 用来帮助你快速完成 P0：
Multi-Agent + Tool Calling + SQL Execution + SQL Repair + Streamlit + Eval。

mallahyari/langgraph-sql-agent 用来帮助你完成 P1/P3：
多 Agent 模块化、图表生成、工程结构、前端展示、SSE / API 化。

azain47/Multi-Agent-Text2SQL-System 保留为局部参考：

sqlglot SQLValidator
Feedback Formatter
iterative feedback loop

最终开发策略：

先用 adamfaik 的思路做出可运行 P0。
再用 azain47 的思路增强 SQL Reviewer。
然后用 mallahyari 的思路扩展图表、报告和工程结构。
P2 的周报、行动计划、审批和告警由 InsightFlow 自己设计。

项目第一原则：

不要杂交太多项目。
只参考最相关的能力点。
最终仍然要服务于 InsightFlow 的核心目标：
Multi-Agent + Tool Calling。